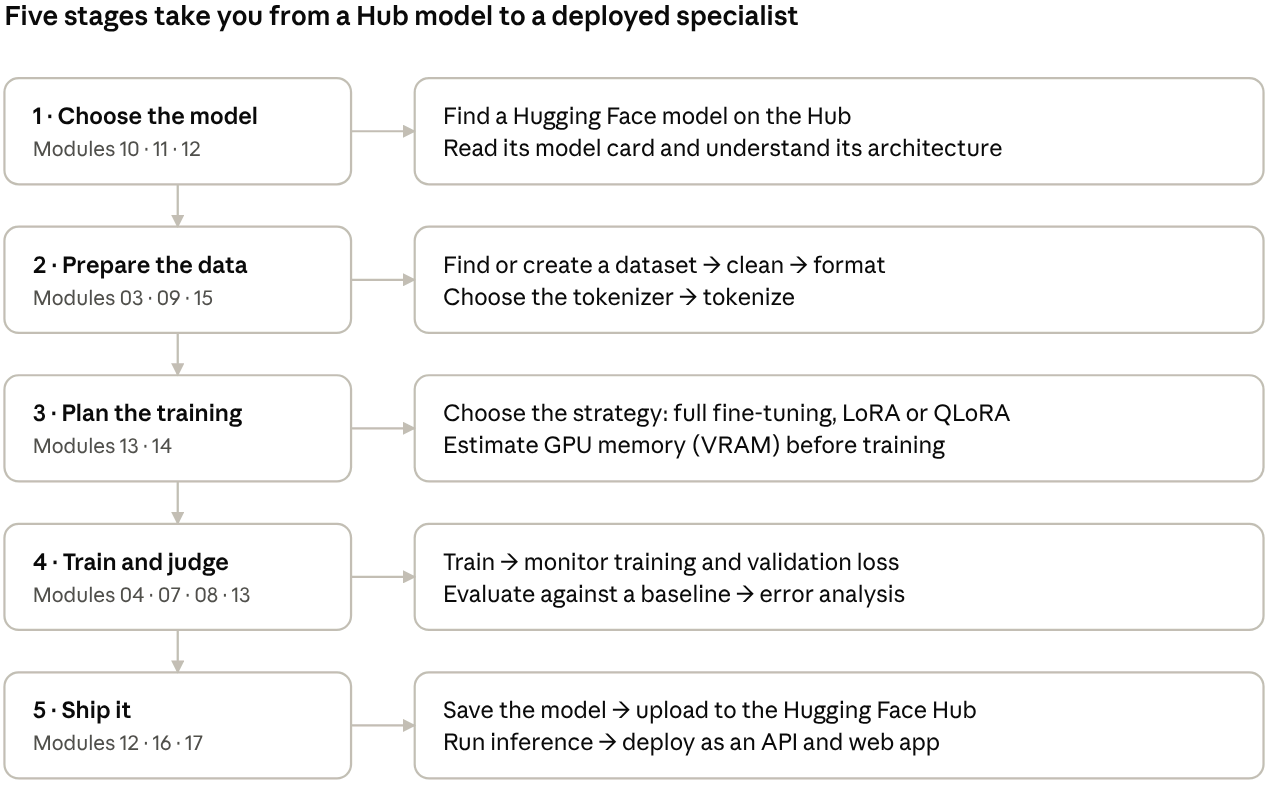
00 — Course Roadmap: Machine Learning to LoRA & QLoRA

Oct 8, 2026 · @Ashishpreet

This roadmap takes you from Python basics to fine-tuning and deploying a Hugging Face model with LoRA and QLoRA in 17 modules, about 455 hours of study. Each module is its own document; this one shows how they fit together.

Final objective

By the last module you will run this whole pipeline on your own, understanding every step rather than copying a notebook.



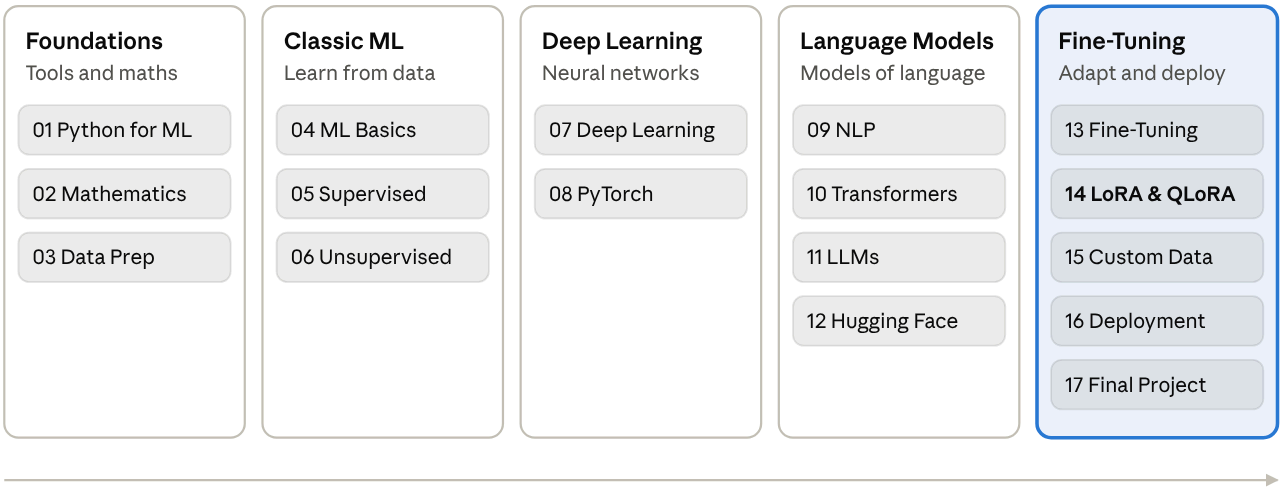
Final objective · 5 stages, 18 steps

Read it top to bottom: each stage needs the one above it, and the grey line under each stage name tells you which modules teach it. Stage 3 is the decision most beginners skip; Module 14 teaches you to make it with numbers.

Course map

The 17 modules fall into five phases; the first four exist to make the fifth, fine-tuning, understandable rather than magic.

Five phases build up to LoRA and QLoRA fine-tuning



Study left to right: each phase uses the tools of the one before it

Course map · 17 modules in 5 phases

How the modules connect: Python (01) carries everything. Maths (02) explains the training loop (04), which explains neural networks (07), which PyTorch (08) implements. Text processing (09) feeds Transformers (10), the architecture inside every LLM (11) you load from Hugging Face (12). Fine-tuning (13) then applies the training loop to those models, LoRA and QLoRA (14) make it affordable, your own dataset (15) makes it useful, and deployment (16) makes it usable. Module 17 joins all of it.

Module guide

Every module answers one question you will face when fine-tuning a model, so nothing here is theory for its own sake. Levels: 🟢 Beginner · 🟡 Intermediate · 🔴 Advanced.

#	Module	Level	You will be able to…	Why it matters for LoRA / QLoRA
01	Python for Machine Learning	🟢	Write Python, use NumPy, Pandas and Matplotlib on real data	Every training script, dataset and evaluation is Python

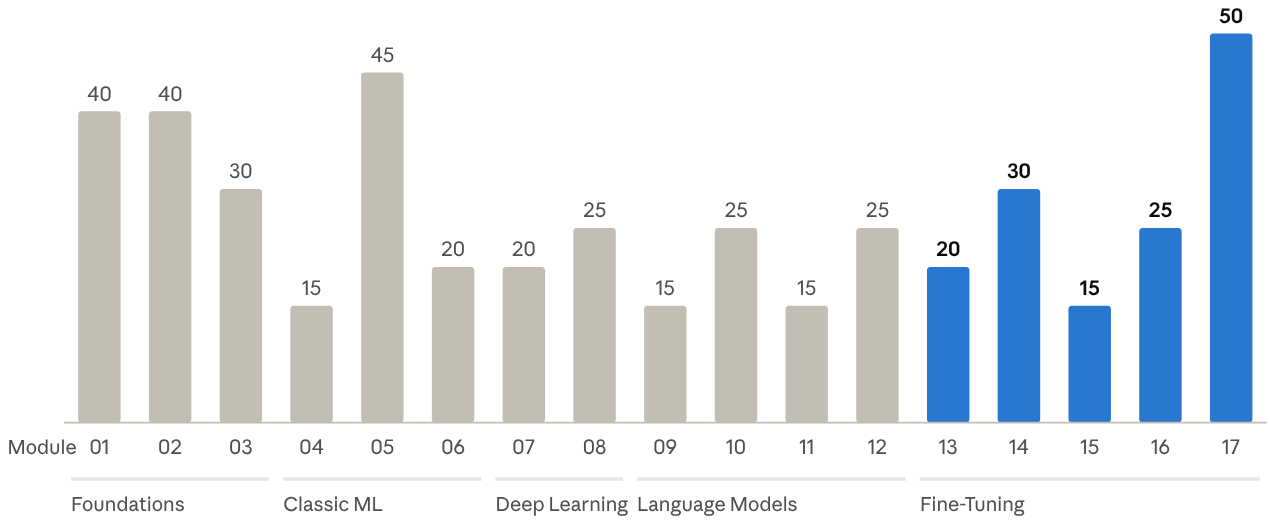
#	Module	Level	You will be able to…	Why it matters for LoRA / QLoRA
02	Mathematics for ML	🟢	Use vectors, matrices, derivatives, gradients, probability	Transformers are matrix multiplications; LoRA is a matrix trick; training is gradient descent
03	Data Preparation	🟢	Clean, explore, visualise and split a dataset without leakage	A fine-tune is only as good as its dataset
04	ML Fundamentals and Model Training	🟢	Explain the loop dataset → prediction → loss → gradient → update	The same loop runs inside every Hugging Face Trainer
05	Supervised Learning	🟡	Build regression and classification models and judge them with the right metrics	Evaluation habits carry straight into judging fine-tuned models
06	Unsupervised Learning	🟡	Cluster data, reduce dimensions, detect anomalies	Embeddings and RAG search rely on the same ideas of distance and similarity
07	Deep Learning	🟡	Explain neurons, layers, activations, backpropagation, optimisers	A Transformer is a deep network; LoRA changes how its layers are trained
08	PyTorch	🟡	Write tensors, Dataset, DataLoader, model, training and validation loops on a GPU	Hugging Face models are PyTorch models
09	NLP	🟡	Turn text into tokens, IDs and embeddings	Tokenisation is the first step of every LLM fine-tune
10	Transformers	🔴	Explain attention, Q/K/V, multi-head attention, encoder vs decoder	LoRA attaches to the attention and feed-forward layers you study here

#	Module	Level	You will be able to…	Why it matters for LoRA / QLoRA
11	Large Language Models	🟡	Explain pretraining, context windows, instruction tuning; choose prompting vs RAG vs fine-tuning	Decides whether fine-tuning is even the right tool
12	Hugging Face	🟡	Find, load, run, save and upload models, tokenizers and datasets	The toolkit for everything that follows
13	Fine-Tuning	🔴	Fine-tune a pretrained model end to end with Trainer	The base workflow LoRA and QLoRA build on
14	LoRA and QLoRA	🔴	Train adapters, quantise to 4-bit, estimate VRAM	The main goal of the course
15	Custom Dataset	🟡	Build instruction and chat datasets in JSONL, good vs bad examples	Your own data is what makes the model yours
16	Model Deployment	🟡	Serve a model via FastAPI, Gradio and Docker; publish to the Hub	A model nobody can use has no value
17	Final Manufacturing AI Project	🔴	Build a manufacturing engineering assistant from documents to web app	Proves you can do every step independently

Status: Documents 00 and 01 are written now; the rest follow one at a time, in order.

Time plan

The full course is about 455 hours: roughly 15 months at 1 hour a day, 7–8 months at 2 hours, or 5 months at 3 hours. The capstone alone is 50 hours, because it repeats every earlier skill on your own data.



Planned study hours per module · estimates, not measurements

Phase	Modules	Weeks at 1 h/day	Weeks at 2 h/day	Weeks at 3 h/day
Foundations	01–03	16	8	5
Classic ML	04–06	11.5	6	4
Deep Learning	07–08	6.5	3	2
Language Models	09–12	11.5	6	4
Fine-Tuning	13–17	20	10	6.5
Whole course	01–17	**~65**	**~33**	**~22**

> **Remember:** these are planning estimates. If Python is already comfortable, Module 01 may take half the time; if maths is rusty, Module 02 may take longer. Speed matters less than ticking every "Can I move on?" box.

Projects ladder

You build nine projects, each one step harder, and each reuses what the one before taught; by the capstone, every step of the pipeline has been practised at least once.

Step	Project	Module	Level	Data	What it proves
1	Should-cost explorer: price vs weight, cycle time, cavities	01	🟢	Synthetic part data	You can load, analyse and plot data in Python
2	Machine speed vs defect rate regression	04–05	🟢	Illustrative process data	You understand the training loop and a regression line
3	Machine failure prediction	05	🟡	AI4I 2020 Predictive Maintenance (UCI)	You can pick a metric and threshold from the cost of errors
4	Sensor anomaly detection and machine-state clustering	06	🟡	Simulated vibration/temperature, then a public bearing dataset	You can find structure without labels
5	Defect-type classifier in PyTorch	07–08	🟡	Simulated process parameters	You can write a full PyTorch training loop on a GPU
6	Shop-floor issue router (text classification)	09, 12–13	🟡	Issue tickets you write or export	You can fine-tune a Hugging Face model and publish it
7	Domain Q&A model with LoRA	14–15	🔴	300+ Q&A pairs from SOPs you know	You can train a LoRA adapter on your own data
8	Same model, larger base, with QLoRA	14	🔴	Same dataset	You can fit a bigger model on a free GPU
9	**Manufacturing Engineering AI Assistant**	16–17	🔴	Your SOPs, WIs, 8D reports	You can go from documents to a deployed web app

Every project ends with a short write-up: problem, data, approach, result, what you would do next. Together they form a portfolio you can show an employer or a university.

Prerequisites, technology stack and environment

You need school maths, basic Python exposure and a Google account; everything else is taught or installed as you go.

Prerequisites

- [ ] Algebra: equations, graphs, exponents, logarithms (your B.Tech covers this)

- [ ] Basic differentiation: slope of a curve (refreshed in Module 02)
- [ ] Some Python exposure: you have run a script before
- [ ] A Google account (for Colab) and a free Hugging Face account
- [ ] A GitHub account to keep your code
- [ ] 1–3 hours a day, plus one longer weekend session for projects

Technology stack

Layer	Tool	First used in	What it does
Language	Python 3.11+	01	Everything is written in it
Arrays and maths	NumPy	01	Fast vector and matrix maths
Tables	Pandas	01	Load, clean and analyse datasets
Plots	Matplotlib, Seaborn	01	Graphs for exploration and results
Classic ML	scikit-learn	04	Regression, classification, clustering, metrics
Deep learning	PyTorch 2.x	08	Tensors, neural networks, GPU training
Models	Hugging Face Transformers 5.x	12	Pretrained models, tokenizers, Trainer
Data	Hugging Face Datasets	12	Load and process training data
Efficient fine-tuning	PEFT	14	LoRA adapters
LLM training	TRL 1.x	14	Supervised fine-tuning (SFTTrainer)
Quantisation	bitsandbytes	14	4-bit and 8-bit models for QLoRA
Serving	FastAPI, Gradio, Docker	16	APIs, web apps, containers

> **Important — versions change.** Hugging Face libraries move fast. Each document states the version its code was checked against and flags version-sensitive lines with ⚠️. As of October 2026: Transformers 5.x (use `dtype=`, not `torch_dtype=`; Trainer takes `processing_class=`) and TRL 1.x.

Recommended environment

1. **Start in Google Colab** (colab.research.google.com): free, runs in the browser, includes a free T4 GPU (Runtime → Change runtime type → T4 GPU). Use it for Modules 01–14.
2. **Save notebooks to Google Drive and GitHub** so nothing is lost when a session ends.
3. **Set up a local environment by Module 16**, when you build apps:

```
# Install Python 3.11+ from python.org, then:
python -m venv ml-env
source ml-env/bin/activate        # Windows: ml-env\Scripts\activate
pip install --upgrade pip
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
# Later modules add: torch transformers datasets peft trl bitsandbytes
accelerate gradio fastapi
```

4. **Editor:** VS Code with the Python and Jupyter extensions.
5. **Hugging Face token:** huggingface.co → Settings → Access Tokens. Never paste it into code you share.

GPU options

A free Colab T4 (16 GB) is enough for every exercise up to QLoRA on a ~4B model; you only pay if you want faster runs or models above ~8B.

Option	GPU (VRAM)	Cost	Can run	Limits
Google Colab, free	T4 (16 GB)	Free	Modules 01–15; LoRA up to ~3B, QLoRA up to ~8B with short sequences	Sessions time out; no bf16
Kaggle Notebooks	T4 ×2 or P100 (16 GB)	Free, weekly GPU quota	Same as Colab, longer runs	Weekly hour cap

Option	GPU (VRAM)	Cost	Can run	Limits
Colab paid tiers	L4 (24 GB) / A100 (40–80 GB)	Monthly fee or compute units	bf16 training, 7–8B LoRA on L4	Units run out
Lightning AI Studios	Various	Free monthly credits, then paid	Persistent setup between sessions	Credit-based
RunPod, Vast.ai	RTX 4090 (24 GB) to H100 (80 GB)	Pay per hour	Large QLoRA runs	You manage the machine
E2E Networks (India)	L4, A100, H100	Pay per hour, billed in INR	Same, with Indian data residency	Setup effort
Own PC	RTX 3060 12 GB / 4060 Ti 16 GB	One-time purchase	Unlimited practice; private company data stays local	Upfront cost

> **Remember:** prices and free quotas change often; check the provider's pricing page before relying on it. Module 14 teaches you to calculate whether a model fits a GPU before you rent one.

How to study with these documents

Read a section, run its code yourself, then do its exercise before reading on; understanding comes from typing and breaking the code, not from reading it.

Every module document has the same layout, so you always know where to look:

1. Why am I learning this? (link to LoRA/QLoRA)
2. Learning objectives and prerequisites
3. Concepts, each taught as: what it is → why we need it → intuition → diagram → maths with real numbers → Python → manufacturing example
4. Common mistakes, interview questions
5. Exercises, mini project, quiz (answers at the very end)
6. Summary, cheat sheet, further learning

7. "Can I move on?" checklist

A study session that works (90 minutes): 10 min re-read yesterday's cheat sheet → 50 min one or two concepts with code → 20 min exercise → 10 min write three lines in your own words about what you learned.

When stuck: re-run the example exactly as written, read the error's last line, change one thing at a time. Then ask me with the code and the error message; I'll explain rather than just fix it.

The "Can I move on?" rule

Only start the next module when you can tick all five for the current one:

- [ ] I understand the concept
- [ ] I can explain it to a colleague without notes
- [ ] I can write the basic code from memory
- [ ] I can interpret the result
- [ ] I can solve a simple new problem with it

If one box stays empty, redo that module's exercises, or ask me to teach that part another way.